```
import java.util.Iterator;

import components.queue.Queue;
import components.queue.Queue1L;
import components.set.Set;
import components.set.SetSecondary;

/**
 * {@code Set} represented as a {@code Queue} of elements with implementations
 * of primary methods.
 *
 * @param <T>
 *            type of {@code Set} elements
 * @convention |$this.elements| = |entries($this.elements)|
 * @correspondence this = entries($this.elements)
 */
public class Set2<T> extends SetSecondary<T> {

    /*
     * Private members --------------------------------------------------------
     */

    /**
     * Elements included in {@code this}.
     */
    private Queue<T> elements;

    /**
     * Finds {@code x} in {@code q} and, if such exists, moves it to the front
     * of {@code q}.
     *
     * @param <T>
     *            type of {@code Queue} entries
     * @param q
     *            the {@code Queue} to be searched
     * @param x
     *            the entry to be searched for
     * @updates q
     * @ensures <pre>
     * perms(q, #q)  and
     * if <x> is substring of q
     *  then <x> is prefix of q
     * </pre>
     */
    private static <T> void moveToFront(Queue<T> q, T x) {
        assert q != null : "Violation of: q is not null";

       Queue<T> temp = q.newInstance();
       boolean hasFound = false;
       T found = null;

       while(q.length()>0) {
           T y = q.dequeue();
           if(!hasFound&&y.equals(x)) {
                   found = y;
                   hasFound = true;
           }
           else {
                   temp.enqueue(y);
           }

       }

       if(hasFound) {
           q.enqueue(found);
       }

       while(temp.length()>0) {
           q.enqueue(temp.dequeue());
```

```
        }
    }

    /**
     * Creator of initial representation.
     */
    private void createNewRep() {
        this.elements = new Queue1L<T>();
    }

    /*
     * Constructors -----------------------------------------------------------
     */

    /**
     * No-argument constructor.
     */
    public Set2() {
        this.createNewRep();
    }

    /*
     * Standard methods -------------------------------------------------------
     */

    @SuppressWarnings("unchecked")
    @Override
    public final Set<T> newInstance() {
        try {
            return this.getClass().getConstructor().newInstance();
        } catch (ReflectiveOperationException e) {
            throw new AssertionError(
                    "Cannot construct object of type " + this.getClass());
        }
    }

    @Override
    public final void clear() {
        this.createNewRep();
    }

    @Override
    public final void transferFrom(Set<T> source) {
        assert source != null : "Violation of: source is not null";
        assert source != this : "Violation of: source is not this";
        assert source instanceof Set2<?> : ""
                + "Violation of: source is of dynamic type Set2<?>";
        /*
         * This cast cannot fail since the assert above would have stopped
         * execution in that case: source must be of dynamic type Set2<?>, and
         * the ? must be T or the call would not have compiled.
         */
        Set2<T> localSource = (Set2<T>) source;
        this.elements = localSource.elements;
        localSource.createNewRep();
    }

    /*
     * Kernel methods ---------------------------------------------------------
     */

    @Override
    public final void add(T x) {
        assert x != null : "Violation of: x is not null";
        assert !this.contains(x) : "Violation of: x is not in this";

        this.elements.enqueue(x);

    }
```

```
    @Override
    public final T remove(T x) {
        assert x != null : "Violation of: x is not null";
        assert this.contains(x) : "Violation of: x is in this";

        moveToFront(this.elements, x);
        T result = this.elements.dequeue();
        return result;
    }

    @Override
    public final T removeAny() {
        assert this.size() > 0 : "Violation of: |this| > 0";
        return this.elements.dequeue();

    }

    @Override
    public final boolean contains(T x) {
        assert x != null : "Violation of: x is not null";
        int i = 0;
        boolean result = false;
        while(i<this.elements.length()&&!result) {
                T y = this.elements.dequeue();
                if(x.equals(y)) {
                        result = true;
                }
                i++;
                this.elements.enqueue(y);
        }
        return result;
    }

    @Override
    public final int size() {
        return this.elements.length();
    }

    @Override
    public final Iterator<T> iterator() {
        return this.elements.iterator();
    }

}
```